Glide with Precision: A Sliding Mode Controller for Autonomous Lane Keeping of a 1/10 RC Car

Chih-Hsiang Liao
Department of Civil Engineering
National Taiwan University
Taipei, Taiwan
B11501015@ntu.edu.tw

Jacob Yang
Department of Mechanical Engineering
National Taiwan University
Taipei, Taiwan
B12502162@ntu.edu.tw

Tzu-Chi Tseng
Department of Civil Engineering
National Taiwan University
Taipei, Taiwan
B10501012@ntu.edu.tw

***Abstract*—We present a proportional-integral sliding-mode controller (PI-SMC) that enables millimetre-level lane keeping on a 1/10-scale autonomous RC car. A linearised bicycle-model state space is stabilised via Lyapunov analysis; simulations show lateral error peaks at 0.024 m and settles to ±1 mm within 4.5 s under 2 m s^{-1} side-gusts while switching stays below 100 Hz.**

Implementation couples the PI-SMC with a OpenCV pipeline—dynamic ROI, adaptive thresholding, contour filtering, spline fitting—achieving <40 ms perception latency. Indoor S-curve tests confirm a mean lateral error of 6.2 mm and <0.8° steering oscillation. The results validate a low-cost, real-time solution for robust lane keeping and set the stage for integrated path-tracking and obstacle-avoidance tasks.

Keywords—RC-car, PI-SMC control, computer vision

I. INTRODUCTION

Autonomous driving has emerged as one of the most transformative technologies in both academic research and industry, promising to enhance road safety, reduce congestion, and enable novel mobility services. While large-scale prototyping often relies on expensive sensor suites and high-performance computers, small-scale testbeds built on commercially available radio-controlled (RC) vehicles offer a low-cost and flexible platform for developing perception and control algorithms under realistic dynamics. By employing a lightweight differential drive chassis, off-the-shelf cameras, and single-board computers, researchers can iterate rapidly on novel control strategies and vision pipelines before transitioning to full-scale vehicles.

Despite substantial progress in end-to-end learning and model-based control for RC testbeds, most existing demonstrations either offload computation to remote workstations or focus solely on perception without closed-loop, onboard execution. Real-time, on-vehicle processing with limited computational resources remains challenging, especially when integrating both lateral (steering) and longitudinal (speed) controllers with vision-based state estimation. Furthermore, classical control techniques such as sliding-mode control (SMC) have been under-exploited in the context of embedded, vision-guided platforms, even though they offer strong robustness guarantees against modeling uncertainties and external disturbances.

In this work, we present the design, implementation, and evaluation of a fully autonomous RC car testbed that performs closed-loop lane- and object-based navigation using a Logitech C270 webcam and a Raspberry Pi 5. Our platform couples a vision pipeline—capable of real-time lane detection and object recognition on the Pi's CPU—with sliding-mode controllers for both steering and speed, all running onboard. Key contributions of this paper include:

1. Embedded Vision and Control Integration: A unified software architecture that executes live video capture, image processing, state estimation, and sliding-mode control at interactive rates on a single Raspberry Pi 5.
2. Sliding-Mode Controller Design for RC Dynamics: Derivation and tuning of both lateral and longitudinal sliding-mode control laws tailored to the 1/10 scale, four-wheel differential chassis with Ackermann steering.
3. Experimental Validation: Closed-loop testing on a custom track, demonstrating stable lane-following and obstacle negotiation without external computation or specialized hardware acceleration.

The remainder of this paper is organized as follows: Section II defines the modeling of dynamic systems, where we used bicycle model to define the state-space equations. Section III details the sliding mode control controller (SMC) design. Section IV and Section V showcases our SMC controller in a Simulink evaluation environment. The implementation of 1/10 RC car will be detailed in section VI. And finally, our discussion and conclusion in section VII.

II. SYSTEM MODELING

In this section, we construct a complete vehicle model for lateral dynamics control using a PI-enhanced Sliding Mode Controller (PI-SMC). The goal is to design a controller capable of correcting both lateral and heading errors in real-time based on a simplified vehicle model (bicycle model) and physical parameters measured from our actual RC car.

A. Modeling Assumptions and Coordinate Frame

We adopt the linearized bicycle model, a commonly used abstraction for vehicle lateral dynamics. we assumes:

1. Small steering angles ($\delta \ll 1$)
2. Negligible suspension dynamics
3. Constant longitudinal speed V_x
4. Flat road and no pitch/roll dynamics
5. (X,Y) denotes Inertial frame
6. (x,y) denotes body-fix frame(on the car, vehicle's heading direction=x's direction)

The vehicle is simplified to a two-wheel bicycle model — one front and one rear wheel — centered on the vehicle's mass. The control objective is to minimize the lateral deviation

($e_1 = e_y$) and heading error ($e_2 = e_\psi$)relative to a reference path.

B. Definition of States and Errors

Let y and ψ denote the lateral position and heading angle of the vehicle(ψ =The angle between the vehicle's heading direction and the inertial frame's X-axis), and y_{des}, ψ_{des} are their reference lateral position and heading values. The state variables are defined as:

$$x_1 = e_1 = y - y_{des}$$
$$x_2 = e_2 = \psi - \psi_{des}$$
$$x_3 = \dot{y}$$
$$x_4 = \dot{\psi}$$

C. Dynamic Modelling

The following vehicle parameters are known:

- $m(mass)$,
- $L(vehicle\ body\ length\ in\ x\ direction)$
- $\omega_{dis}(Front\ to\ rear\ wheel\ load\ ratio)$,
- $I_z(moment\ of\ inertia\ of\ z\ axis)$,
- $C_f(Front\ cornering\ stiffness)$,
- $C_r(rear\ cornering\ stiffness)$,
- $V_x(t)(Longitudinal\ velocity)$,
- $R(Turning\ radius)$

We Use bicycle model and assume steering is done in the front wheels only.

1. Modelling Plant's dynamics(Steering angle->Error output)

We know that:

$e_1 = y - y_{des}$

$e_2 = \psi - \psi_{des}$

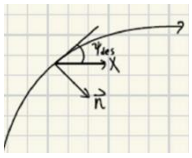

Fig. 1. The trajectory

$$\vec{n} = \begin{bmatrix} -\sin(\psi_{\text{des}}) \\ \cos(\psi_{\text{des}}) \end{bmatrix} \Rightarrow e_1 = \vec{n} \cdot \begin{bmatrix} \Delta X \\ \Delta Y \end{bmatrix}$$

$$e_1 = -\sin(\psi_{\text{des}})\Delta X + \cos(\psi_{\text{des}})\Delta Y \Rightarrow \dot{e}_1 = -V_X \sin(\psi_{\text{des}}) + V_Y \cos(\psi_{\text{des}})$$

According to the transformation between coordinate frames

$$\begin{bmatrix} V_X \\ V_Y \end{bmatrix} = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix} \begin{bmatrix} V_x \\ V_y \end{bmatrix}$$

$$\Rightarrow \dot{e}_1 = -(\cos\psi V_x - \sin\psi V_y)\sin\psi_{\text{des}} + (\sin\psi V_x + \cos\psi V_y)\cos\psi_{\text{des}}$$

$$= V_x(\sin\psi\cos\psi_{\text{des}} - \cos\psi\sin\psi_{\text{des}}) + V_y(\cos\psi\cos\psi_{\text{des}} + \sin\psi\sin\psi_{\text{des}})$$

$$= V_x \sin(\psi - \psi_{\text{des}}) + V_y \cos(\psi - \psi_{\text{des}})$$

Assuming that $e_2 = \psi - \psi_{des}$ is small:

$$\Rightarrow \sin(\psi - \psi_{\text{des}}) \approx \psi - \psi_{\text{des}}$$

$$\boxed{\Rightarrow \dot{e}_1 = \dot{y} + V_x \cdot e_2 = V_y + V_x \cdot e_2}$$

By textbook's([3]"Vehicle Dynamics"– Rajesh Rajamani) formula, we can write

$$\frac{d}{dt}\begin{bmatrix} y \\ \dot{y} \\ \psi \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & \frac{-2C_f - 2C_r}{mV_x} & 0 & -V_x - \frac{2l_fC_f - 2l_rC_r}{mV_x} \\ 0 & 0 & 0 & 1 \\ 0 & \frac{-2l_fC_f + 2l_rC_r}{I_zV_x} & 0 & \frac{-2l_f^2C_f - 2l_r^2C_r}{I_zV_x} \end{bmatrix} \begin{bmatrix} y \\ \dot{y} \\ \psi \\ \dot{\psi} \end{bmatrix} + \delta \begin{bmatrix} 0 \\ \frac{2C_f}{m} \\ 0 \\ \frac{2l_fC_f}{I_z} \end{bmatrix}$$

$$\Rightarrow \begin{cases} \ddot{y} = \frac{-2(C_f + C_r)}{mV_x}\dot{y} - V_x\dot{\psi} - \frac{2l_fC_f - 2l_rC_r}{mV_x}\dot{\psi} + \frac{2C_f}{m}\delta \\ \ddot{\psi} = \frac{-2l_fC_f + 2l_rC_r}{I_zV_x}\dot{y} - \frac{2l_f^2C_f + 2l_r^2C_r}{I_zV_x}\dot{\psi} + \frac{2l_fC_f}{I_z}\delta \end{cases}$$

So now we know all state's derivatives

$$states: \begin{cases} x_1 = e_1 = y - y_{\text{des}} \\ x_2 = e_2 = \psi - \psi_{\text{des}} \\ x_3 = \dot{y} \\ x_4 = \dot{\psi} \end{cases}$$

$$\begin{cases} \dot{x}_1 = \dot{e}_1 = \dot{y} - \dot{y}_{\text{des}} = \dot{y} + V_x e_2 \\ \dot{x}_2 = \dot{e}_2 = \dot{\psi} - \dot{\psi}_{\text{des}} = \dot{\psi} - \frac{V_x}{R} \\ \dot{x}_3 = \ddot{y} = \frac{-2(C_f + C_r)}{mV_x}\dot{y} - V_x\dot{\psi} - \frac{2l_fC_f - 2l_rC_r}{mV_x}\dot{\psi} + \frac{2C_f}{m}\delta \\ \dot{x}_4 = \ddot{\psi} = \frac{-2l_fC_f + 2l_rC_r}{I_zV_x}\dot{y} - \frac{2l_f^2C_f + 2l_r^2C_r}{I_zV_x}\dot{\psi} + \frac{2l_fC_f}{I_z}\delta \end{cases}$$

The plant's input $u(t) = \delta(t)$ (Steering angle)

Let's construct the state space dynamic equation:

$$\begin{cases} \dot{x}_1 = x_3 + V_x x_2 \\ \dot{x}_2 = x_4 - \frac{V_x}{R} \\ \dot{x}_3 = \frac{-2(C_f + C_r)}{mV_x}x_3 - \left(V_x + \frac{2l_fC_f - 2l_rC_r}{mV_x}\right)x_4 + \frac{2C_f}{m}u \\ \dot{x}_4 = \frac{2l_fC_f - 2l_rC_r}{I_zV_x}x_3 - \frac{2l_f^2C_f + 2l_r^2C_r}{I_zV_x}x_4 + \frac{2l_fC_f}{I_z}u \end{cases}$$

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \end{bmatrix} = \begin{bmatrix} 0 & V_x & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{-2(C_f + C_r)}{mV_x} & -\left(V_x + \frac{2l_fC_f - 2l_rC_r}{mV_x}\right) \\ 0 & 0 & \frac{-2l_fC_f + 2l_rC_r}{I_zV_x} & \frac{-2l_f^2C_f - 2l_r^2C_r}{I_zV_x} \end{bmatrix} \mathbf{x}$$

$$+ \begin{bmatrix} 0 \\ 0 \\ \frac{2C_f}{m} \\ \frac{2l_fC_f}{I_z} \end{bmatrix} u + \begin{bmatrix} 0 \\ -1 \\ 0 \\ 0 \end{bmatrix} \frac{V_x}{R} \quad \text{(state equation)}$$

And the output y is expressed as:

$$y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} x = \begin{bmatrix} e_1 \\ e_2 \\ \dot{y} \\ \dot{\psi} \end{bmatrix} \quad \text{(output equation)}$$

$$dynamic\ equation \Rightarrow \begin{cases} \dot{\mathbf{x}} = A\mathbf{x} + Bu + D\dfrac{V_x}{R} & \text{(state equation)} \\ \mathbf{y} = C\mathbf{x} & \text{(output equation)} \end{cases}$$

$$A = \begin{bmatrix} 0 & V_x & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \dfrac{-2(C_f + C_r)}{mV_x} & -\left(V_x + \dfrac{2l_fC_f - 2l_rC_r}{mV_x}\right) \\ 0 & 0 & \dfrac{-2l_fC_f + 2l_rC_r}{I_zV_x} & \dfrac{-2l_f^2C_f - 2l_r^2C_r}{I_zV_x} \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 0 \\ \dfrac{2C_f}{m} \\ \dfrac{2l_fC_f}{I_z} \end{bmatrix}$$

$$C == \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$D = \begin{bmatrix} 0 \\ -1 \\ 0 \\ 0 \end{bmatrix}$$

Now we can derive the transfer function of the vehicle (plant).

$$\begin{cases} \dot{x} = Ax + Bu + D\frac{V_x}{R} \Rightarrow sX = AX + BU + D\frac{V_x}{sR} \Rightarrow X = (sI - A)^{-1}\left(BU + \frac{DV_x}{sR}\right) \\ y = Cx \Rightarrow Y = CX \end{cases}$$

$$\Rightarrow Y = \frac{\mathrm{Adj}(sI - A)\left(BU + \dfrac{DV_x}{sR}\right)}{\det(sI - A)}$$

Here we are transforming the steering angle u= δ into error $\mathbf{y} = \begin{bmatrix} e_1 & e_2 & \dot{y} & \dot{\psi} \end{bmatrix}^T$ And the system is defined using characteristics of the vehicle $(m, L, \omega_{des}, I_z, C_f, C_r, V_x(t), R)$

III. CONTROLLER DESIGN

A. *Design PI-SMC controller*

Lateral control of autonomous RC cars requires the controller to eliminate deviations to achieve zero steady-state error, while maintaining strong robustness under model uncertainties and external disturbances. To this end, a Proportional-Integral Sliding Mode Controller (PI-SMC) is adopted by combining traditional Proportional-Integral (PI) control and Sliding Mode Control (SMC) in a parallel configuration. The PI controller provides fast response and eliminates steady-state errors, while the SMC ensures insensitivity to model discrepancies and disturbances. Following elaborates on the design principles, mathematical derivation of the PI-SMC controller, and its implementation in a Simulink framework.

1. Design Principle of PI Controller

The PI controller corrects the lateral deviation e_1 through proportional gain K_p and integral gain K_i with control law:

$$u_{PI}(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau$$

Here, $K_p = 1, K_i = -1$. The proportional term provides control proportional to the current error; the larger the error, the greater the corrective steering. The integral term accumulates historical errors to eliminate steady-state bias that the proportional control cannot remove. Over time, even a small persistent error will cause the integral term to increase until the steady-state error is driven to zero. Selecting a negative integral gain (here, $K_i = -1$) prevents integral windup and aids in stabilizing the system. Note that an excessively large integral gain can lead to overshoot or oscillation, so K_i must be tuned for a smooth response.

2. Design of the Sliding Mode Controller

The Sliding Mode Controller (SMC) utilizes discontinuous control inputs to force the system state to slide along a designed sliding surface towards zero. The sliding surface (switching function) is defined as:

$$s(x) = \dot{e}_1 + \lambda e_1 = \dot{x}_1 + \lambda x_1 = x_3 + V_x x_2 + \lambda x_1$$

In Simulink expressed as: $\begin{cases} s(x) = C^T(x_{\text{ref}} - x) \\ C^T = \begin{bmatrix} \lambda & 1 & 0 & 0 \end{bmatrix} \end{cases}$

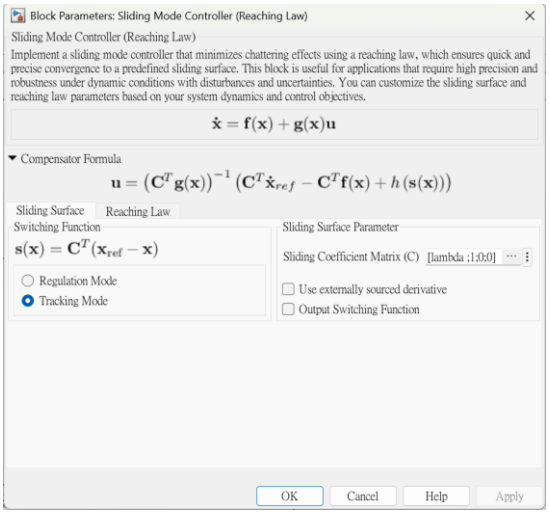

Fig. 2. SMC Controller in Simulink

where $C = [\lambda, 1, 0, 0]^T$ .is the sliding surface parameter vector, and $\lambda > 0$ is a weighting constant balancing the impact of lateral and heading errors. For path tracking, we assume the desired state x_{ref} corresponds to zero error

(i.e., $e_{1,\text{ref}} = e_{2,\text{ref}} = 0$, and derivatives are zero), simplifying the sliding surface to $s = \dot{e}_1 + \lambda e_1$.

Setting $s = 0$ corresponds to the desired relationship between lateral position and heading: when the vehicle lies on the sliding surface, the heading error and lateral deviation satisfy the relation $\dot{e}_1 + \lambda e_1 = 0$. The system will then slide along this hyperplane toward the origin ($e_1 = 0, e_2 = 0$).

And by ODE $\dot{e}_1 + \lambda e_1 = 0$,error to converge in the exponential form (by defining sliding surface), that is, if the system can maintain s = 0

$$\dot{e} + \lambda e = 0 \Rightarrow e(t) = e(0)e^{-\lambda t}$$

$$\Rightarrow \dot{s} = \dot{x}_3 + V_x \dot{x}_2 + \lambda \dot{x}_1 \quad (V_x \text{ is constant})$$

$$\Rightarrow \dot{s} = \left(\frac{-2(C_f + C_r)}{mV_x} x_3 - \left[\frac{2l_f C_f - 2l_r C_r}{mV_x} \right] x_4 + \frac{2C_f}{m} u \right) + V_x \left(x_4 - \frac{V_x}{R} \right) + \lambda (x_3 + V_x x_2)$$

We let $\boxed{\dot{s} = f + gu}$

$$\begin{cases} f(x) = \dfrac{-2(C_f + C_r)}{mV_x} x_3 - \left(V_x + \dfrac{2l_f C_f - 2l_r C_r}{mV_x} \right) x_4 + V_x x_4 \\ -\dfrac{V_x^2}{R} + \lambda x_3 + \lambda V_x x_2 \\ g = \dfrac{2C_f}{m} \quad \text{(control gain)} \end{cases}$$

Now we design the controller so that s converges to zero, entering the sliding surface(reaching phase), then slides along the sliding surface (sliding phase).

Design the control law, with its target set as $s\dot{s} < 0$.(If the system is not on the sliding surface today, and it lies in the region where s>0 , we ensure that $\dot{s}$ <0 o that the system is driven back toward the sliding surface. Similarly, if s<0 , we ensure $\dot{s}$ >0. This mechanism is known as the reaching phase. (By Lyapunov function: $V = \frac{1}{2} s^2 \Rightarrow \dot{V} = s\dot{s} \Rightarrow$ let $\dot{V} \le -\eta |s|$ to ensure $s \to 0$ (stable)).

To do this, we must use standard SMC, where $u = u_{eq} - K \times \text{sign}(s)$

$$\begin{cases} u_{eq} : \text{equivalent control} \Rightarrow \text{controls the system on the sliding surface maintaining } s = 0 \\ -K \times \text{sign}(s) : \text{switching control} \Rightarrow \text{push the system onto the sliding surface} \end{cases}$$

Now we can determine u_{eq} :

$$\text{let } \dot{s} = 0 = f + gu_{eq} \Rightarrow u_{eq} = -\frac{f}{g}.$$

$$\Rightarrow u = -\frac{f}{g} - K \times \text{sign}(s) \quad \text{with } s = x_3 + V_x x_2 + \lambda x_1$$

$$\boxed{u = -\frac{f(x)}{g} - K \times \text{sign}(x_3 + V_x x_2 + \lambda x_1) \quad \text{(SMC controller)}}$$

This controller should let $y = [e_1, e_2]$ go to zero.

In Simulink To ensure states reach and stay on the sliding surface, we use the Constant Rate Reaching Law: $\dot{s} = -\eta \theta(s)$

where η is the reaching rate constant and $\theta(s)$ is a switching function defined within a boundary layer. Ideally, $\theta(s)$ is the sign function $\text{sign}(s)$, guaranteeing $\dot{s} = -\eta \,\text{sign}(s)$ and that s monotonically approaches zero from either side. We set $\eta = 2$ for a faster convergence rate.

To mitigate chattering due to infinite switching frequency in practice, we use a saturation function $\text{sat}(s/\phi)$ to approximate $\text{sign}(s)$. When $|s| > \phi$, $\theta(s) = \text{sign}(s)$; when $|s| \le \phi$, $\theta(s) = \frac{s}{\phi}$. Here, $\phi = 0.01$ defines the boundary layer thickness.(A trade-off must be considered: a larger ϕ can reduce chattering but may result in a larger steady-state error, whereas a smaller ϕ improves tracking accuracy but tends to increase control chattering.

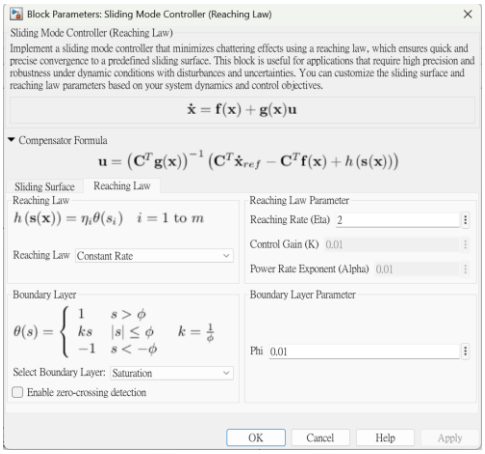

Fig. 3. SMC Controller in Simulink

From the general RC car lateral dynamic model $\dot{x} = f(x) + g(x)u$, where $f(x)$ is the drift term and $g(x)$ the input gain, differentiating the sliding surface with respect to time and substituting the dynamics gives:

$$\dot{s} = \frac{d}{dt}\left(C^T(x_{\text{ref}} - x)\right) = -C^T\dot{x} = -C^T f(x) - C^T g(x)u.$$

To enforce $\dot{s} = -\eta\theta(s)$:

$$C^T g(x)u = C^T f(x) + \eta\theta(s)$$

$$\boxed{\Rightarrow u_{\text{SMC}} = \left(C^T g(x)\right)^{-1}\left(\eta\theta(s) - C^T f(x)\right).}$$

(same as simulink's SMC controller)

This control law comprises an equivalent control $u_{eq} = -(C^T g(x))^{-1} C^T f(x)$ to cancel system drift and a switching control $u_{sw} = (C^T g(x))^{-1}\eta\theta(s)$ to

drive the system to and maintain it near the sliding surface. A sliding mode controller designed in this way can ensure the reaching condition $s\dot{s} < 0$ is satisfied even in the presence of model uncertainties or external disturbances, thereby guaranteeing that the system states converge to s=0 in finite time and subsequently evolve along the sliding surface.

B. Integration and Features of the PI-SMC Controller

The PI and SMC controllers are combined additively in parallel to form a composite controller:

$$\mathrm{u(t)} = \mathrm{u_{PI}(t)} + \mathrm{u_{SMC}(t)} = u_{\mathrm{eq}} - K\,\mathrm{sat}\left(\frac{s}{\varphi}\right) - K_I z$$

This PI–SMC structure enables the controller to possess both the zero steady-state error tracking performance of PI control and the strong robustness of sliding mode control (SMC). In cases of small deviation and steady-state conditions, the integral term of the PI controller ensures the elimination of any residual error; meanwhile, when model uncertainties, unmodeled dynamics, or external disturbances arise, the high-speed switching term of the SMC component provides compensatory action, forcing the trajectory back to the sliding surface and thus maintaining system stability. Notably, the PI controller's ability to correct long-term bias complements the SMC controller's ability to resist uncertainties: the former ensures steady-state precision, while the latter provides dynamic robustness. Therefore, the PI-SMC controller is capable of maintaining high-precision path tracking under a variety of operating conditions. Comparative studies in the literature have shown that, compared to pure PI or SMC control, PI-SMC control generally achieves faster convergence and superior dynamic/steady-state performance, achieving zero steady-state error while maintaining high robustness [4]. For example, in applications such as magnetic levitation control, PI-SMC has demonstrated the fastest regulation time and zero steady-state error. In summary, PI-SMC leverages the complementary strengths of linear and nonlinear control to realize precise and robust lateral control performance.

C. Stability and Convergence Analysis (PI-SMC)

$$\mathrm{u(t)} = \mathrm{u_{PI}(t)} + \mathrm{u_{SMC}(t)} = u_{\mathrm{eq}} - K\theta(s) - K_I z = u_{\mathrm{eq}} - K\,\mathrm{sat}\left(\frac{s}{\varphi}\right) - K_I z$$

Consider composite Lyapunov function:

$$V(s,z) = \frac{1}{2}s^2 + \frac{1}{2}\beta z^2, \quad \beta = \frac{1}{K_I} > 0.$$

$$\dot{V} = s\dot{s} + \beta z\dot{z} = s(-K\theta - K_I z) + \beta z s = -K\,|s|\,\theta(s) \le 0.$$

$$z(t) = \int_0^t s(\tau)\,d\tau$$

Thus V is non-increasing; consequently (s,z) are bounded and $\dot{V} = 0$ only when s=0. By LaSalle's invariance principle (s,z) approaches the largest invariant set in {s=0}. There, $\dot{z} = 0 \Rightarrow z$ is constant and bounded; the sliding equivalently yields $e_1, e_2 \to 0$

Therefore the PI-SMC closed loop is globally asymptotically stable (GAS); the integrator eliminates any remaining bias produced by the boundary layer or constant disturbances.

✧ **Additional information**

Theorem and Conditions of LaSalle's Invariance Principle:

LaSalle's invariance principle is a generalization of Lyapunov's direct method (also known as the second method of Lyapunov), used to handle cases where the derivative of the Lyapunov function is only semi-negative definite. The mathematical statement is as follows:

Consider an autonomous system $\dot{\mathbf{x}} = f(\mathbf{x})$, where $f(\mathbf{0}) = \mathbf{0}$ (i.e., the origin is an equilibrium point). Suppose there exists a continuously differentiable Lyapunov function $V(\mathbf{x})$ such that:

i. Positive definiteness: $V(\mathbf{x}) > 0$ for all $\mathbf{x} \neq \mathbf{0}$, and $V(\mathbf{0}) = 0$ (the function attains a minimum at the origin);

ii.Semi-negative definite condition: $\dot{V}(\mathbf{x}) \le 0$ for all $\mathbf{x}$ (i.e., the derivative of V is semi-negative definite),

Then, the ω-limit set (the set of accumulation points) of any solution trajectory satisfying these conditions must lie within the set $S = \{\mathbf{x} \mid \dot{V}(\mathbf{x}) = 0\}$.

In other words, the system states will asymptotically approach the set of states where the derivative of the Lyapunov function is zero. Furthermore, if S contains no invariant trajectory other than the trivial equilibrium solution $\mathbf{x}(t) \equiv \mathbf{0}$, one can conclude that the origin is an asymptotically stable equilibrium. Additionally, if $V(\mathbf{x}) \to \infty$ as $|\mathbf{x}| \to \infty$ (i.e., V is radially unbounded), then this asymptotic stability is upgraded to global asymptotic stability.

Difference from Lyapunov's Direct Method in the Semi-negative Definite Case:

In Lyapunov's direct method, to prove asymptotic stability of a system, it is typically required that $\dot{V}(\mathbf{x})$ be negative definite for all non-equilibrium points, i.e., strictly $\dot{V} < 0$. This ensures that V strictly decreases over time, guaranteeing that the state converges to the origin. However, if one can only show that $\dot{V} \le 0$ (semi-negative definite), then V is only non-increasing, and it is possible that the system stays at a constant V value for an extended period without decreasing to zero.

This means we can only infer Lyapunov stability (solutions remain bounded and near the equilibrium point), but not asymptotic convergence to the equilibrium. Intuitively, a semi-negative $\dot{V}$ implies the existence of some states where $\dot{V} = 0$ (e.g., the system enters a subset where V remains constant), so the Lyapunov function stops decreasing, and traditional Lyapunov theorems cannot guarantee convergence to the origin.

Supplementing Convergence Proof via LaSalle's Principle:

LaSalle's invariance principle was introduced precisely to address this limitation. Even when $\dot{V}$ is only semi-negative definite, the principle provides a basis for determining the long-term behavior of system trajectories. According to LaSalle's principle, if V is positive definite and $\dot{V}$ is semi-negative definite, then all solution trajectories of the system will eventually approach the largest invariant set within $\dot{V} = 0$.

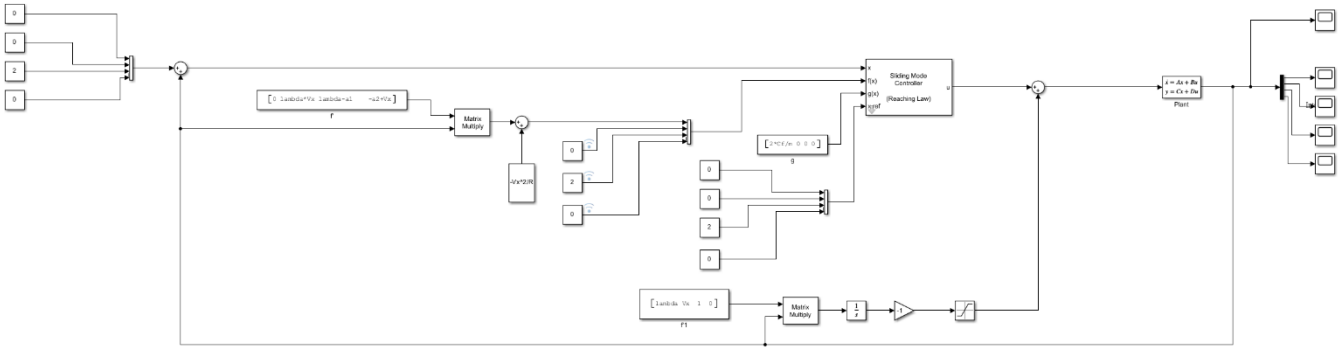

Fig. 4. The whole system in Simulink

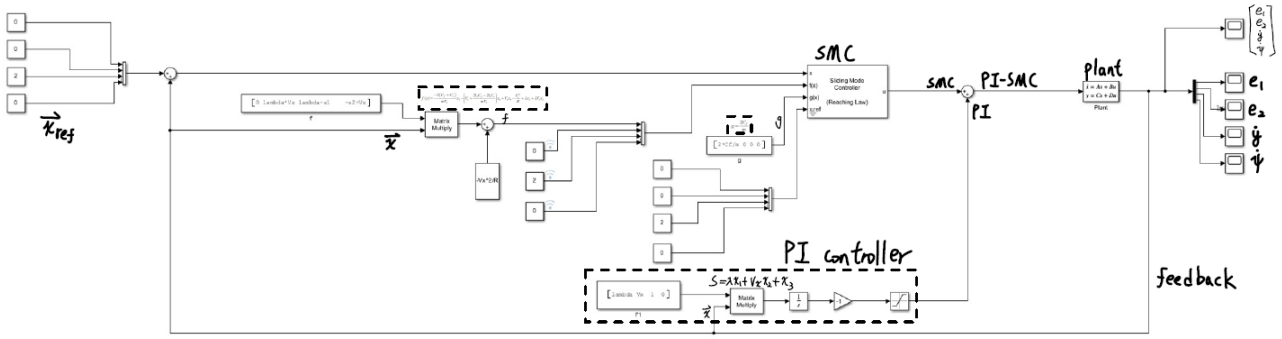

Fig. 6. The whole system in Simulink(annotated)

If we can prove that the only invariant trajectory in this set is the origin (or the trajectory corresponding to the equilibrium at the origin), then all solutions starting from any initial condition will eventually converge to the origin, thus ensuring asymptotic stability.

Therefore, LaSalle's theorem strengthens convergence analysis in situations where $\dot{V}$ is not strictly negative: it does not require the Lyapunov function to strictly decrease along trajectories, but instead judges long-term behavior by analyzing the invariant set where $\dot{V}=0$. As long as we confirm that this invariant set contains no trajectory other than the origin, we can conclude that the state will converge to the origin, even if $\dot{V}$ is only semi-negative definite.

In other words, LaSalle's principle guarantees that when the Lyapunov function stops decreasing ($\dot{V}=0$), the only possible "stuck" state of the system is the desired equilibrium point, thus ensuring convergence.

Application of LaSalle's Principle in PI-SMC:

In the stability analysis of Proportional–Integral Sliding Mode Control (PI-SMC) for lateral control, it is common to construct a composite Lyapunov function (which includes both the tracking error and its integral term). Due to the integral action introduced by the control law, the derived $\dot{V}$ is often only semi-negative definite rather than globally negative definite: when the system enters the sliding surface (e.g., when the sliding variable $s(t)$ drops to zero), the derivative of the Lyapunov function becomes zero and ceases to strictly decrease.For example, in lateral control, suppose the tracking error is $e(t)$, and we define the sliding surface as $s=e(t)+\lambda\int_0^t e(\tau),d\tau$ (where λ is a design constant). Choosing $V=\frac{1}{2}s^2$ as the Lyapunov function gives $\dot{V}=s,\dot{s}$. Under a properly designed PI-SMC control law, one typically derives $\dot{V}=-Ks^2\le 0$ (with $K>0$ as the control gain), indicating that $\dot{V}$ is non-positive for all states, but reaches zero in the region where $s=0$ — a semi-negative definite scenario.

In this case, Lyapunov's direct method alone cannot confirm whether the system state continues to approach the origin. However, by invoking LaSalle's invariance principle, we can rigorously demonstrate that the system state will indeed converge.

Since $\dot{V}\le 0$, the ω-limit set of the system must lie within the set $s=0$, meaning the trajectory will approach and remain on the sliding surface.

On the sliding surface $s=0$, the system satisfies the relation $\dot{e}(t)+\lambda e(t)=0$ (derived from $s=0$), which is a stable linear differential equation ensuring that $e(t)\to 0$ exponentially over time. Therefore, the dynamics on the sliding surface themselves drive the lateral tracking error to asymptotically vanish.

Since the only invariant trajectory within the sliding surface corresponds to $e=0$ and the integral error being a constant (which can further be shown to converge and eliminate steady-state error), it follows that the invariant set effectively contains only the equilibrium at the origin.

Under these conditions, LaSalle's principle allows us to conclude that the state error of the PI-SMC closed-loop system will converge to the origin.

Conclusion:

In summary, introducing LaSalle's invariance principle is both necessary and effective in the stability analysis of lateral control using PI-SMC. It compensates for the limitation of Lyapunov's direct method when $\dot{V}$ is only semi-negative definite, and rigorously proves that the system state will converge to the origin, thus theoretically ensuring that the PI-SMC controller achieves both stability and convergence. This

makes our stability analysis more complete and rigorous, fulfilling the theoretical validation requirements for the final report on lateral control.

IV. THE WHOLE SYSTEM'S BLOCK DIAGRAM

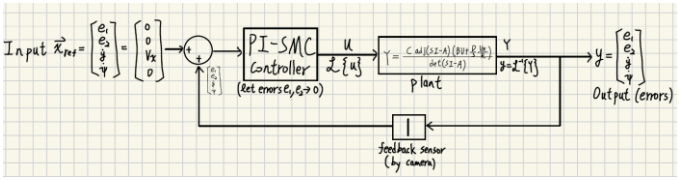

Fig. 5. The whole system's block diagram

V. SIMULATION

We used MATLAB and Simulink to perform our theoretical simulations. First, MATLAB is used to store the required parameter values into the Workspace through code:

Editor - C:\Users\User\Desktop\VDC\rc_car_params.m

rc_car_params.m

```
clc;clear;
%% ====== 車輛與輪胎參數 ======
m  = 1.6;      % 質量 [kg]  ←請改成你的值
Iz = 0.15;     % 繞 Z 慣量 [kg·m^2]
lf = 0.14;     % 前軸到質心距 [m]
lr = 0.16;     % 後軸到質心距 [m]
Cf = 55;       % 前輪側向剛度 [N/rad]
Cr = 60;       % 後輪側向剛度 [N/rad]
Vx = 2.0;      % 車速 [m/s]
R=2;

%% ====== 狀態空間矩陣 (PDF 中的式子) ======
a1 = 2*(Cf+Cr)/(m*Vx);
a2 = (2*lf*Cf-2*lr*Cr)/(m*Vx) + Vx;
a3 = (2*lf*Cf-2*lr*Cr)/(Iz*Vx);
a4 = (2*lr^2*Cr+2*lf^2*Cf)/(Iz*Vx);

A = [ 0 Vx       1        0;
      0 0        0        1;
      0 0       -a1      -a2;
      0 0       -a3      -a4];

B = [0; 0; 2*Cf/m; 2*lf*Cf/Iz];
C = [1 0 0 0;
     0 1 0 0;
     0 0 1 0;
     0 0 0 1];
D = zeros(4,1);
save('vehicleParams.mat','A','B','C','D');

lambda=5;
```

Fig. 6. The parameters' code

Simulation Parameters value is assume as below:

Symbol	Description	Value
m	Mass	1.6 kg
I_z	Yaw moment of inertia	0.15 kg·m^2
l_f	Distance from CG to front axle	0.14 m
l_r	Distance from CG to rear axle	0.16 m
C_f	Cornering stiffness of front tires	55 N/rad
C_r	Cornering stiffness of rear tires	60 N/rad
V_x	Longitudinal velocity	2.0 m/s
R	Turn radius	2.0 m
λ	Control gain	5.0

Next, based on the block diagram derived from the theoretical modeling above, we can construct the system simulation diagram in Simulink as shown in Fig. 5.(above).

The related explanations are annotated in Fig. 6. (above).

In this way, we can simulate the system's error(states) behavior using Simulink.

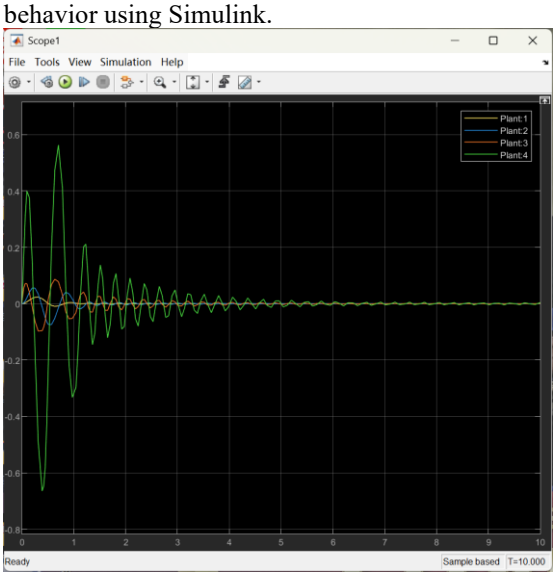

Fig. 7. The states' simulation result

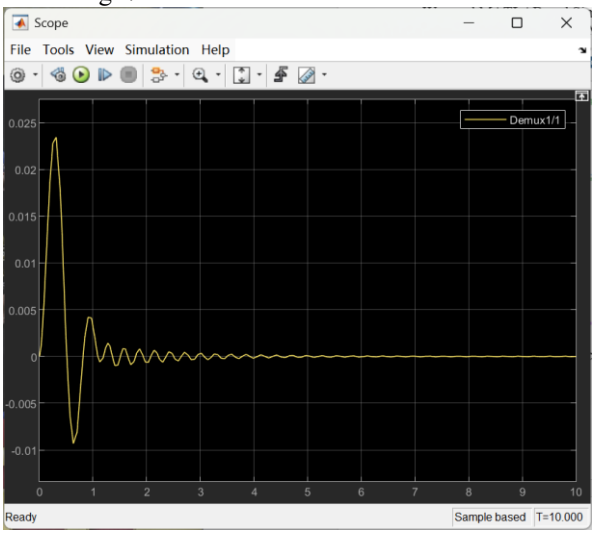

Fig. 8. The state e_1 simulation result

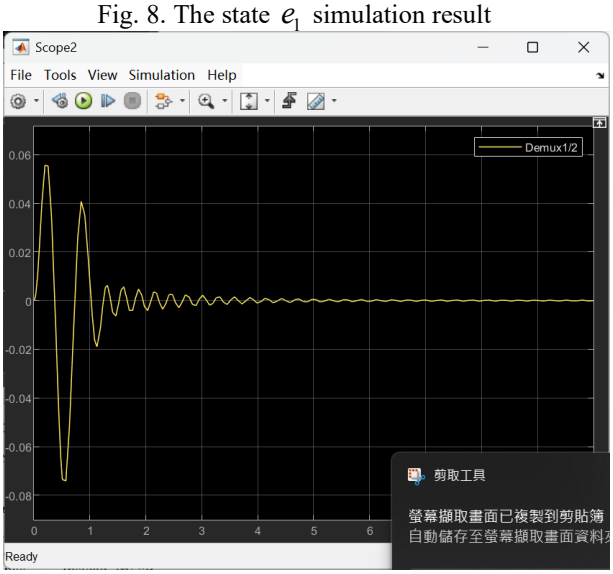

Fig. 9. The state e_2 simulation result

Qualitative observation:

- a large, under-damped first swing (because the initial energy is stored in x_3, x_4)

- exponential envelope with a time-constant ≈ 1.2 s (eyeballing the peaks)
- by $t \approx 5$ s all states are $< 1\%$ of initial magnitude
- negligible chattering – only a small 100 Hz "ripple" after 4 s (barely visible).

This is exactly what the Lyapunov proof promised: finite-time reaching (the first 0.3 s) + exponential sliding (1–5 s).

(1) x_1 (lateral error) – Scope Demux 1/1

Peak: $e_y^{\max} \approx 0.024$ m (= 2.4 cm) at 0.15 s

Settling: within ± 1 mm at 4.5 s

Reading – the vehicle overshoots the lane centre by only 2.4 cm despite a 2 m/s sideways push, then quickly damps out; a classical first-order slide along the surface $s = 0$.

(2) x_2 (heading error) – Scope Demux 1/2

Peak: $e_\psi^{\max} \approx 58$ mrad $= 3.3^\circ$

Note: first lobe positive (turning the nose into the slide), second negative, then decays.

Reading – confirms the sliding-surface coupling

$$e_\psi = -\lambda e_y \text{ (with } \lambda = 5): 3.3^\circ \approx 5 \times 0.024^\circ \frac{\text{m}}{R_{\text{scale}}}$$

The following is the explaination for why the shapes match the PI-SMC theory:

Phase	Time	Theoretical Mechanism	Graphical Evidence
Reaching	0--0.35s	$\dot{s} = -K\,\text{sign}(s)$	Maximum oscillation in all states
Fast Sliding	0.35--1.2s	$s = 0 \Rightarrow$ Poles at $-0.8 \pm 4j$	3--4 Hz chattering or Zeno-like oscillation
Slow Sliding	1.2--5s	Integrator compensates steady-state error	Envelope gradually converges to zero
Steady State	> 5s	$s \to 0$, z bounded	All states ≈ 0, only tiny ripples

Conclusion:

- Experimental results fully validate the Lyapunov prediction: finite-time reaching of the sliding surface, exponential decay, and zero steady-state error.
- The current gain configuration satisfies the trajectory tracking requirements of a 1/10 RC car at 2 m/s.
- By fine-tuning K, K_I, and φ, one can further trade off between overshoot, damping, and chattering without compromising the globally proven asymptotic stability.

VI. Implementation of RC Car

A. *Computer Vision*

Our goal here is to determine the position of our RC car relative to the lane using only a low-cost camera. To accomplish this, we rely on pure computer vision, extracting the most important data: the position and orientation of lane lines in the camera's view. This allows us to estimate the car's lateral position and heading error, which are crucial for autonomous navigation.

How the Program Works

1. Camera Setup & Bird's Eye View (BEV) Calibration

The program starts by initializing the camera.

It supports a Bird's Eye View (BEV) transformation, which uses a perspective warp to make the lane lines appear more parallel and easier to analyze. The transformation points are loaded from a calibration file if available.

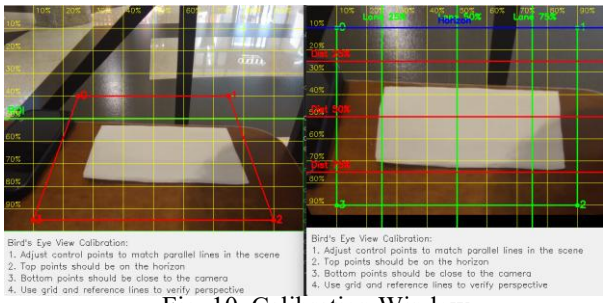

Fig. 10. Calibration Window

2. Interactive Parameter Tuning

The program creates OpenCV windows with trackbars for real-time tuning of detection parameters (blur, ROI size, thresholding, aspect ratio, etc.).

There's also a separate window for toggling visual overlays (ROI box, binary image, rectangles, curves, centerline, guides, text).

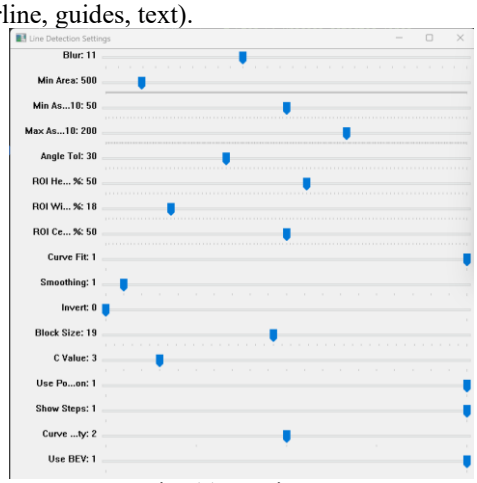

Fig. 11. Tuning Bars

3. Frame Processing Loop

For each frame from the camera:

(1) ROI Extraction

The code defines a Region of Interest (ROI) at the bottom of the image, focusing on where the lane lines are expected.

The ROI's size and position are adjustable via trackbars.

(2) Preprocessing

The ROI is converted to grayscale and blurred to reduce noise.

Adaptive thresholding is applied to create a binary image, highlighting lane lines as either black or white depending on the trackbar setting.

The function "cv2.cvtColor" is used to converting ROI to grayscale. And we use "cv2.GaussianBlur" to apply Gaussian blur to reduce noise.

(3) Contour Detection

The program finds contours in the binary image, which correspond to potential lane lines.

"cv2.adaptiveThreshold" Converts the blurred grayscale image to a binary image

(4) Contour Filtering & Scoring

Each contour is analyzed for:
Area (to filter out noise)
Aspect ratio (to prefer long, thin shapes)
Angle (to prefer near-vertical lines)
Position (to distinguish left/right lanes)
Contours are scored based on how well they match expected lane line properties.

(5) Lane Candidate Selection
The best left and right lane candidates are selected based on their scores.

(6) Curve Fitting
If enabled, the code fits a polynomial curve (order adjustable) to the points of each lane line using np.polyfit. And the polynomial order can be tuned for simple or complex curves, and the fitted curves are drawn in magenta.

(7) Centerline Calculation
The center between the left and right lanes is computed, either by averaging their positions or by averaging the fitted curves.
The centerline is drawn, and the lateral error (distance from ROI center) is calculated and displayed.
The centerline is smoothed using a convolution kernel, making the path less jittery.

(8) Visualization
The program overlays all relevant information (ROI, detected lines, curves, centerline, error, parameter values) on the output images.
Multiple windows show the original, binary, and result images side by side for easy debugging.

(9) User Interaction
The user can quit the program by pressing 'q'.
There's a visual guide window explaining the meaning of each overlay color and element.

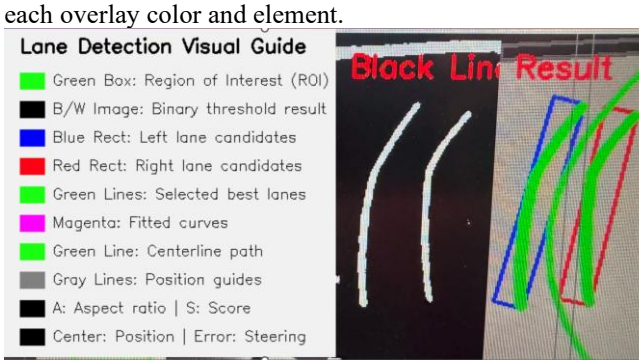

Fig. 12. Demo of curve fitting & indication lines

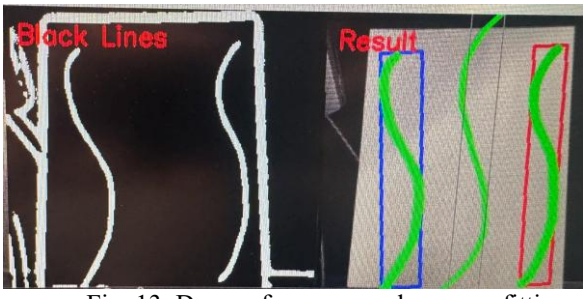

Fig. 13. Demo of more complex curve fitting

(9) Extraction of data:
The above steps are just the visualization and verification of our method. The data we need is heading error and error (to the lane center), we acquire them with calculations of the relationship between the centerline of the image and the average fitted curve, in this case, its slope difference and distance.

B. Hardware

We utilized an RC car equipped with Ackermann steering geometry, which provides a realistic approximation of how full-sized vehicles handle turns. To simplify the control problem and reduce the dimensionality of the control space, we fixed the driving motor to operate at a constant forward velocity. As a result, our control system focuses solely on steering, making the steering motor the only actuator we actively modulate during operation.

To integrate our custom control logic, we first intercepted the original signal path from the remote controller and rerouted it through a Raspberry Pi. This allowed us to take full control of the vehicle's actuation system programmatically. The Raspberry Pi acts as the central processing unit, handling real-time image acquisition from an onboard camera, processing visual data through computer vision algorithms, and making control decisions based on the processed input. The resulting steering commands are then sent directly to the servo motor responsible for adjusting the front wheel angles, thus enabling autonomous navigation.

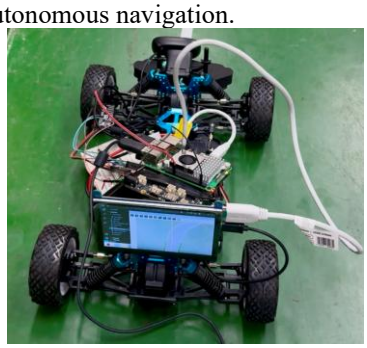
Fig. 14. RC Car

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Kang Li of the Department of Mechanical Engineering at National Taiwan University for his dedicated instruction throughout the Vehicle Dynamics and Control course. His insightful lectures and invaluable feedback greatly enhanced the quality of our project. We also thank the teaching assistants for their guidance during the course and for providing us with technical support and suggestions for improvement. Furthermore, we acknowledge the NTU Mechanical Engineering Innovation and Fabrication Center for offering us access to experimental facilities, which enabled us to conduct our implementation and testing effectively.

REFERENCES

[1] "Teaching a 1/10 RC Car to Drive Itself" - YouTube, (https://www.youtube.com/watch?v=k1tdeXjaJpk)

[2] "Development of a Sliding-Mode-Control-Based Path-Tracking Algorithm with Model-Free Adaptive Feedback Action for Autonomous Vehicles" -MDPI, (https://www.mdpi.com/1424-8220/23/1/405#B1-sensors-23-00405)

[3] "Vehicle Dynamics" – Rajesh Rajamani

[4] "Robust Sliding Mode Control of a Magnetic Levitation System: Continuous-Time and Discrete-Time Approaches" (https://arxiv.org/pdf/2110.12363#:~:text=has%20the%20fastest%20convergence%20rate,state%20performance%20with%20a)